

Day 19 - Lecture 19.9: Practice Task (Weekly Temperature Analysis)

1. Project Overview & Objective

In this project, we analyze and visualize the weekly temperature trends across four major global cities: **New York**, **London**, **Delhi**, and **Tokyo** across weekdays (Monday through Friday).

Key Learning Objectives:

- Applying the **Object-Oriented Subplots** workflow in a practical scenario.
 - Mastering `axes.flatten()` to iterate over multi-dimensional subplot grids cleanly in a single `for` loop.
 - Using shared figure-level labels (`fig.suptitle` , `fig.supxlabel` , `fig.supylabel`).
 - Implementing nested loop alternative approaches and understanding their trade-offs.
-

2. Grid Architecture & Array Flattening

When creating a 2×2 grid:

```
fig, axes = plt.subplots(2, 2)
```

The returned `axes` is a 2D NumPy array of shape `(2, 2)` : $\begin{bmatrix} \text{ax}[0,0] & \text{ax}[0,1] \\ \text{ax}[1,0] & \text{ax}[1,1] \end{bmatrix}$

Why `axes.flatten()` ?

A nested loop requires:

```
count = 0
for i in range(2):
    for j in range(2):
        axes[i, j].plot(...)
        count += 1
```

By calling `axes = axes.flatten()`, the 2D array is transformed into a 1D array of 4 elements:

$$[ax_0, ax_1, ax_2, ax_3]$$

This allows a much cleaner and Pythonic loop using `enumerate()`:

```
for i, ax in enumerate(axes.flatten()):
    ax.plot(...)
```

3. Implementation Code

```
import matplotlib.pyplot as plt

days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
cities = ["New York", "London", "Delhi", "Tokyo"]

temperatures = [
    [22, 23, 21, 24, 25], # New York
    [18, 19, 17, 20, 21], # London
    [30, 32, 31, 33, 34], # Delhi
    [25, 26, 24, 27, 28] # Tokyo
]

# Approach 1: Modern Pythonic loop with axes.flatten()
fig, axes = plt.subplots(2, 2, figsize=(10, 8), sharey=False)
axes_flat = axes.flatten()

colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728']

for i, ax in enumerate(axes_flat):
    ax.plot(days, temperatures[i], marker='o', linewidth=2, color=colors[i], label=cities[i])
    ax.set_title(cities[i], fontsize=13, fontweight='bold')
    ax.grid(True, linestyle='--', alpha=0.6)
    ax.set_ylim(15, 38) # Common scale for fair visual comparison

# Figure-level global labels
fig.suptitle("Weekly Temperature Trends Across Global Cities", fontsize=16, fontweight='bold')
fig.supylabel("Temperature (°C)", fontsize=13)
fig.supxlabel("Day of the Week", fontsize=13)

fig.tight_layout()
plt.show()
```

4. Key Takeaways

1. **`fig.supxlabel()` and `fig.supylabel()`** : Replaces repetitive axis labels on every individual subplot, giving a cleaner, publication-grade appearance.
2. **Unified Axis Scales**: Notice that setting `ax.set_ylim(15, 38)` across all panels allows immediate visual comparison of absolute temperatures (e.g. noticing Delhi is significantly hotter than London).

